DEVELOPING A PROGRAM FOR SIMULATING THE BUILDING BLOCKS OF LIFE

Matura Thesis by Jonathan Goth

JONATHAN GOTH, 4A, GYMNASIUM BÄUMLIHOF
SUPERVISOR: DR. ANDREAS LEUTHARDT
CO-EXAMINER: ADRIAN KAISER
BASEL, 10/11/2026

Table of Contents

1 Introduction 3
2 Process 5
2.1 Programming language 5
2.2 Graphics Tools 5
2.3 Graphics Section 5
2.4 Simulation Section 6
2.5 Accessibility Features 7
2.6 AI Usage 7
3 Product 8
3.1 Elements of the Simulation 8
Particle 8
Compound 8
3.2 Global Variables 9
3.3 Simulation Surface 9
3.4 Global Clock 9
3.5 Forces 11
3.6 Structure 12
3.7 Installation 15
3.8 Compiling and Executing 15
3.9 Future 15
4 Capabilities of the Program 16
4.1 Phospholipid 16
Typ 1 16
Typ 2 17
4.2 DNA/RNA 18

4.3 Protein 19
4.4 Usage 20
5 Limits/Problems 20
6 Conclusion 21
7 Glossary 22
8 Sources 23
8.1 Online Repository 23
8.2 Websites 23
8.3 Image Sources 25
8.4 AI 25
9 Acknowledgments 25
10 Statement of Integrity 26

1 Introduction

I've been interested in the natural sciences since I was a child, and for the past few years, I've also been interested in programming. I'm particularly fascinated by combining these two interests and using the resulting programs to try to simulate physical, chemical, and biological processes. While researching such simulations, I came across the concept of "emergence." It describes the emergence of complex properties in a system whose individual components do not possess those properties. Because of its unpredictability, it is one of the most frequently simulated phenomena, with many different examples. Two of them interest me in particular and inspired me to write this program.

The Game of Life (LifeWiki contributors, 2025), developed in 1970 by mathematician John Conway (Bhargava, Gabai, Kochen, & Kohn, n.d.), is probably the best-known and simplest example of emergence. It consists of an infinitely large two-dimensional grid of squares of equal size. Each square represents a cell and can be in one of two states: alive or dead. The formation, survival, and death of a cell depend on the states of its (eight) neighbors:

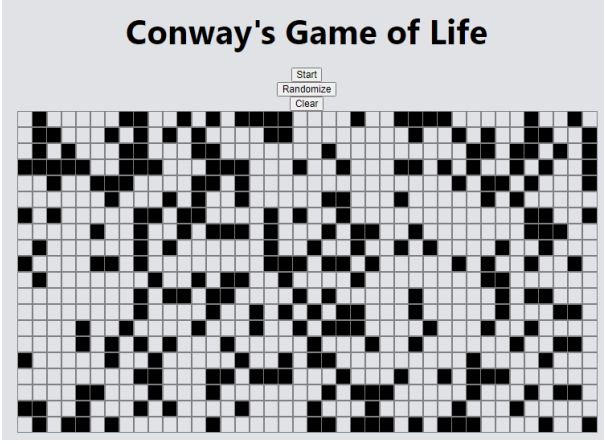


Figure 1: Conway's Game of Life: Black cells are alive; white cells are dead

1. < 2 living neighbors → dead.
2. > 4 living neighbors → dead.
3. 2 living neighbors → remains alive if it is alive (otherwise dead).
4. 3 living neighbors → living.

The simulation proceeds in steps—or, more precisely, generations, with the living cells of one generation depending on those of the previous generation. These simple rules can give rise to enormous, organic-looking structures that grow explosively and, in most cases, collapse just as quickly shortly thereafter. Another characteristic of the Game of Life is that a single, misplaced cell can destroy an entire system within just a few generations.

The second example is Particle Life (Mohr, Particle Life Simulator, n.d.), which was developed in 2022 by software developer Tom Mohr (Mohr, About Me, n.d.) based on "Clusters" (Ventrella, An asymmetrical particle system, 2025) by digital artist Jeffrey Ventrella (Ventrella, jeffrey ventrella, n.d.). It also consists of a two-dimensional plane, which is, however, bounded, and on which "particles" (colored circles) move freely. These particles are divided into groups based on their color, which in turn define their interactions. For this purpose, each of the n groups has n scalars (one for each group → n × n matrix) that define the strength of the interaction. Based on this value, a particle experiences a force when it reaches a certain distance from other particles. These forces can be attractive or repulsive and do not have to be symmetrical. This means that attraction-attraction, repulsion-repulsion, and attraction-repulsion interactions with equal or different magnitudes are possible. With the right values and enough time, structures can emerge in Particle Life that resemble life forms such as single-celled organisms or worms in both their appearance and behavior.

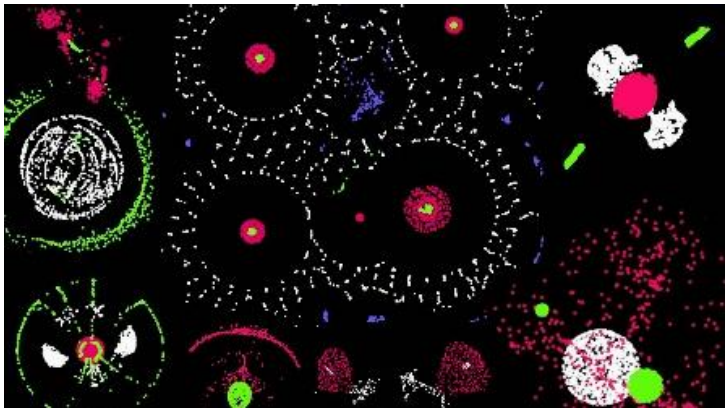


Figure 2: A compilation of some structures that can form in Particle Life

These two programs inspired me to create something similar.

The goal of my program was to simulate organic components as realistically as possible, within certain limitations. At the same time, I ultimately wanted to use it to try to simulate very rudimentary cells with multiple components.

2 Process

2.1 Programming language

Once it had been decided which features the program should have, it became clear that computing power would likely be the most important resource.

The first—and perhaps most important—decision in this regard was choosing which programming language to use. With maximum efficiency as the goal, only a handful of languages were in the running, namely: Assembly, C, C++, Rust, Zig, and Odin. However, some of them were quickly ruled out. These were: Assembly, because the code would have become too complex and would not have been compatible with all systems; and Zig and Odin, because both are too new and their documentation and range of applications are still very limited. Of the remaining languages, C++ was the best choice, as it (along with C) has long been the standard for graphical programs and games/simulations and was ideally suited due to its object-oriented (terms highlighted in blue are explained in the glossary, Chapter 7) capabilities. Furthermore, I already had over 1.5 years of experience with C++ and did not want to learn yet another new programming language.

2.2 Graphics Tools

Since the simulation was intended to include only simple geometric shapes, an existing, well-documented graphics library would have been overkill and might have slowed it down. For this reason, the graphics portion of the program was ultimately written from scratch, in the hope of adding only the necessary elements and avoiding the loss of processing power due to unused code. With an existing library, this could happen if it is designed for various applications and contains code that is not needed for the simulation but is executed nonetheless. The choice ultimately fell on the Vulkan graphics API (Khronos Group, n.d.-b). Released in 2016 by the Khronos Group (Khronos Group, n.d.-a), it quickly grew to become one of the most widely used graphics APIs for games, game engines, and simulation software, and its high efficiency and compatibility make it perfectly suited for this program. The window in which the simulation runs must also be created and linked to Vulkan. GLFW (Geelnard & Löwy, n.d.) was originally used for this purpose. However, while attempting to find a bug in the program, it was replaced by SDL3 (libsdl-org, n.d.).

2.3 Graphics Section

The first phase of development focused on the program's graphics component and lasted about three months, beginning in late January 2026. The code was based on Alexander Overvoorde's Vulkan tutorial (Overvoorde, 2016), which, however, was already outdated and no longer conformed to the current Vulkan standard. Therefore, continued use of it is not recommended. Nevertheless, upon its completion toward the

end of April 2026, the graphics system was able to render tens of thousands of circles without any issues. At the start of development, the graphics component was divided into two classes. The `VulkanHandle` class abstracts interactions with Vulkan—such as initialization—and is intended to make it easier to work with. It is also designed to detect and output errors that occur during Vulkan initialization, which makes it easier to find errors in the graphics component. The second class, `Window`, is responsible for more specific actions, such as drawing shapes, and interacts with a `VulkanHandle` instance for this purpose. Most of the Vulkan tutorial was incorporated into the `VulkanHandle` class with only minor modifications. Afterward, lists for the elements to be drawn were integrated into the `Window` class. Each list stores elements of a specific shape, and initially, lists for circles and rectangles were implemented—which, as it turned out, were all the shapes the simulation needed. The idea behind this is that all shapes to be drawn are temporarily stored and then passed to Vulkan as a group, rather than doing this individually for each shape. The reason for this is that this method is very efficient and should run considerably faster than the alternative. The `DrawCircle()` and `DrawRectangle()` methods of the `Window` class add a new element to the corresponding list, and `Update()` draws all elements in the lists and then clears them.

2.4 Simulation Section

The actual simulation phase began in May and was largely complete by the end of June. In the beginning, it was not yet clear how detailed the simulation of the biological elements should be. In discussions with Mr. Leuthardt, it became clear that it would make sense to simulate them in a very abstract way, focusing only on their form and function. This would simplify the development of the program and significantly reduce the computational load for chemical processes. At this point, the program was already capable of simulating individual particles and smaller compounds, which could be moved around at the mouse cursor's position using a modified particle. The first goal—which ultimately took up the first few weeks—was to simulate phospholipids. The basic shape and movement were quickly implemented, but there were still problems with interaction, especially when two lipids came into contact. The problem with the initial approaches was that attractive forces became stronger as the distance decreased and also acted on lipids that were already in contact with each other. Combined with collision detection, this caused the phospholipids to begin rotating and prevented them from bonding correctly. The solution was to use a force-calculation method in which forces no longer act on lipids that are touching. After some research, the Lennard-Jones potential (Wikipedia contributors, 2026)—an approximation for calculating interactions between non-chemically bonded atoms or particles—was used, which solved the problem. In the final week before the program's scheduled completion, DNA, RNA, proteins for transcription and translation, and the ability to create additional elements were finally added. To improve the simulation's usability, a save system was created that can save and load the simulation's current state as a JSON file. In addition, a particle was added that

allows users to delete elements instead of moving them. From that point on, the simulation was fully functional.

2.5 Accessibility Features

In early October, shortly before the final deadline, two major changes were made to the program to make it clearer and easier to use. First, the process of modifying and adding compounds was simplified. Previously, the program had to be recompiled after every change to the compounds, which required extra software and technical knowledge. The ability to define compounds in an external file—which the program reads—was added, thereby circumventing this issue. To apply changes, the program simply needs to be restarted. The second user aid is an interactive menu in the terminal. This allows certain parameters to be displayed or modified both when launching the program and while it is running. This makes it possible to modify the simulation without having to stop it every time, as was previously the case.

2.6 AI Usage

ChatGPT was used as an aid throughout the entire development process of the program. However, its use was strictly limited to explaining new concepts, demonstrating the use of a C++ feature or library using an example, or finding particularly stubborn bugs in the code. Without this help, the development of the program would have taken much longer, and the simulation would not have been possible on the current scale. It is important to note, however, that all of the program's code was either self-written or taken from a tutorial.

When writing the documentation, ChatGPT was used twice to provide consistent explanations of certain terms. Both sections were manually reviewed, and the use of ChatGPT is indicated in each case (Chapters 3.5 and 7). In addition, DeepL was used to translate this version of the documentation from German to English, and everything was reviewed here as well

3 Product

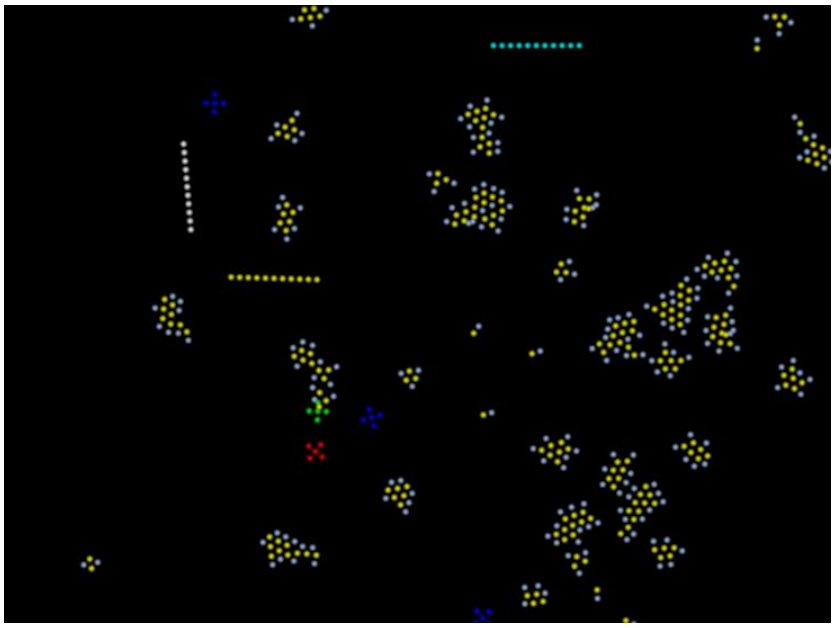


Figure 3: A random example from the simulation using various compounds

3.1 Elements of the Simulation

The elements of the simulation are divided into two groups:

Particle

Particles are the smallest building blocks of the simulation; they consist of a circle with a radius (in nanometers) and a color. They can represent individual atoms or small molecules and are either hydrophilic or hydrophobic. Every particle in the simulation is part of a compound.

Compound

Compounds are groups of particles that form a functional unit, such as a molecule, protein, enzyme, or DNA/RNA. In this context, the structure remains static, meaning the particles occupy fixed positions relative to one another. Compounds interact with other compounds through hydrophilic and hydrophobic interactions and, upon physical contact, can perform predefined functions, which enables new interactions.

3.2 Global Variables

To simplify the organization of the code into multiple files, the saving of a simulation, and the modification of simulation parameters, many of the variables required throughout the entire runtime are implemented as global variables. This means that they can be accessed and modified from anywhere in the code.

3.3 Simulation Surface

The simulation is two-dimensional and limited to a specific size. This area is framed in white and has a default size of 100 nanometers by 100 nanometers. Its position and size are specified by the global variable `simArea`. This is a four-dimensional vector whose components are interpreted as follows:

X → The x-coordinate of the upper-left corner of the surface.
Y → The y-coordinate of the upper-left corner of the surface.
Z → The width of the surface in the positive x-direction.
W → The height of the surface in the positive y-direction.

It follows that the first two components determine the position of the upper-left corner of the surface, and the others determine its size.
During the simulation, the system checks whether a compound is still within the simulation surface based on the center of mass of all its particles. If this is not the case, the system attempts to push the particle back into the simulation area by reversing the x- or y-component of its velocity. This creates a bounce-back effect as soon as a particle touches a wall of the area. If a particle with zero velocity is outside the area, it remains there until it is set in motion from the outside or deleted.

3.4 Global Clock

In computer programs, there are two different time frames that run partially independently of one another. The first is from the observer's perspective—that is, the time required to calculate a frame—and the second is from the program's perspective—that is, the time simulated during that frame. For a simulation running at 10 FPS (frames per second), 0.1 seconds must elapse within the program for each frame so that both time frames are equal.
To depict the processes as realistically as possible, time within the simulation must elapse linearly. This means that the same amount of time must elapse in every frame. Since the time required to calculate a frame increases with growing complexity, the program's speed appears to change. This is normally compensated for by adjusting the simulation time based on the computation time, but it also leads to uneven time jumps

in the simulation and is therefore unsuitable here. Because of that, the simulation time does not correspond to real-world time, which is why it is also impossible to measure the elapsed time in the simulation from the outside. To solve this problem, the simulation has its own clock that measures the simulation time.

By default, the simulation time per frame is set to 0.001 seconds (1 millisecond) and is specified by the global variable `simFrameTime`. Thus, 1 second has elapsed in the simulation after 1,000 frames. To calculate the time after any given frame, the number of frames that have elapsed since the start of the simulation must be counted; this is handled by the global variable `frameCounter`. Using these two variables, the elapsed simulation time can be calculated:

$$frameCounter \times simFrameTime = Elapsed\ simulation\ time$$

However, it is also possible to change the frame time, which makes it more difficult to measure the simulation time. Therefore, every time the frame time is changed, the elapsed time since the last change (or since the start of the simulation, if the frame time has not yet been changed) is measured and added to any previous measurements. This makes it possible to calculate the time elapsed since the start of the simulation up to the current frame only. To measure the time between any two frames using this method, the times of both frames must be known, which means that the elapsed time of the first frame must be measured and stored until the second frame is reached. The global clock enables this through several methods:

`recordNow()` → Must be called in the first frame and saves the current time when the measurement begins.

`read(frame)` → Can be called as often as desired and returns the time elapsed since the specified frame. However, the frame must have been saved beforehand using `recordNow()`.

`deleteEntry(frame)` → Deletes the saved frame to free up memory space when it is no longer needed.

3.5 Forces

The only forces simulated between molecules are hydrophilic and hydrophobic interactions. The strength of the interaction is calculated using the Lennard-Jones potential (Wikipedia contributors, 2026), an approximation for calculating interactions between atoms or particles that are not chemically bonded.

$$V = 4\varepsilon\left\{\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right\} = \varepsilon\left\{\left(\frac{r_m}{r}\right)^{12} - 2\left(\frac{r_m}{r}\right)^{6}\right\}$$

Figure 4: The Lennard-Jones Potential as a Function

Explanation of the parameters generated by ChatGPT (OpenAI, n.d.):

V = potential energy of the interaction between the two particles

→ Indicates how strongly the particles attract or repel each other.

r = current distance between the two particles

→ The interaction energy changes depending on r.

ε = depth of the potential well or strength of the interaction

σ = characteristic distance of the Lennard-Jones potential

r_m = equilibrium distance or distance at the minimum of the potential

→ Here, the potential energy is at its lowest, and the resulting force is zero.

The Lennard-Jones potential describes the potential between two particles and can be converted into a force by differentiation. The curve of the original function depends on the distance between the two particles and results in a negative potential (typically attractive) at a greater distance and a strongly positive potential (typically repulsive) at too small a distance. In between, there is a point at which the potentials balance each other out and no force acts on the particles, which describes their optimal distance.

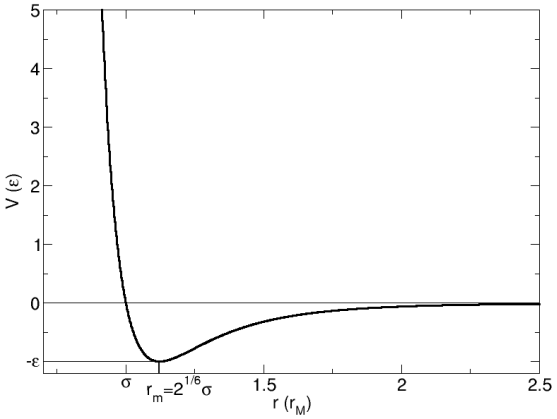


Figure 5: The Lennard-Jones potential plotted as a curve

3.6 Structure

Aside from Vulkan, the entire simulation code and all required libraries are organized in the main folder named `LifeLike-Engine`.

```
LifeLike-engine/
|-- assets/
|   |-- compound_templates.cpp
|-- doc/
|   |-- Doxyfile
|-- include/
|   |-- assets/
|   |   |-- compound_templates.hpp
|   |-- core/
|   |   |-- result.hpp
|   |   |-- types.hpp
|   |   |-- vector.hpp
|   |   |-- vector.inl
|   |-- graphics/
|   |   |-- vulkan_handle.hpp
|   |   |-- vulkan_handle.inl
|   |   |-- window.hpp
|   |-- sim/
|   |   |-- colors.hpp
|   |   |-- compound.hpp
|   |   |-- console_interface.hpp
|   |   |-- functions.hpp
|   |   |-- parameters.hpp
|   |   |-- particle.hpp
|   |   |-- save_load.hpp
|   |   |-- time_manager.hpp
|-- shaders/
|-- src/
|   |-- main.cpp
|   |-- vulkan_handle.cpp
|   |-- window.cpp
|-- third_party/
|-- CmakeLists.txt
|-- LICENSE
|-- README.md
```

`assets/compound_templates.cpp:`

Contains all compounds that can be created in the simulation and all functions that can be executed when two compounds come into contact. By editing this file and compiling it, existing compounds can be modified and new ones created.

`doc/:`

Intended to use Doxygen here to generate documentation for the entire code base.

`include/assets/:`

Contains headers for all C++ files in the `assets/` directory, in which all important elements are declared.

`include/core:`

Contains a header-only library with elements that are used throughout the program. The most important ones are:

Integer data types [`u8_t`, `i8_t`, `u16_t`, `i16_t`, `u32_t`, `i32_t`, `u64_t`, `i64_t`, `size_t`]

Floating-point data types [`f32_t`, `f64_t`]

Boolean data type[`b8_t`]

C-style string [`cstr_t`]

Result enum for debugging and tracking errors in the graphics section [`Result`]

Minimalist n-dimensional vector class [`Vec<2, T>`, `Vec2<T>`, `Vec3<T>`, ...]

`include/graphics:`

Contains headers for the graphics section, in which the classes `VulkanHandle` and `Window` and their members are declared.

`include/sim:`

Contains the headers for the simulation section, in which all remaining elements of the simulation are defined. The declarations and definitions are included in a single file here to make it easier for the compiler to optimize the program.

`shaders:`

Contains the shaders required by Vulkan, which are written in GLSL and have been compiled to SPIR-V.

`src/main.cpp:`

Contains the `int main()` function, which starts the program's execution, and also includes the simulation's main loop, which is repeated in every frame.

`src/vulkan_handle.cpp & src/window.cpp:`

Includes the definitions from the header files in `include/graphics`.

`third_party:`

Includes all libraries used except for Vulkan. These are SDL3 (libsdl-org, 2026), dujingnin's inicpp library (dujingning, 2026), and Niels Lohmann's json library (Lohmann, 2026). It also includes vit-vit's CTPL library (vit-vit, 2015) for parallel code execution for optimization purposes; however, it has been removed (possibly temporarily) from the code to avoid problems.

`CMakeLists.txt:`

Contains the program's CMake configuration.

LICENCE:

The MIT License (MIT, n.d.) under which the program was released and may be used.

`README.md:`

Contains all the necessary information to install and use the program.

3.7 Installation

Updated information and installation instructions can be found in `README.md`.

As of release, the program has only been tested on Windows 11 Home and Arch Linux. Compatibility with macOS, however, has not been tested. It has several dependencies that must be installed:

- The Vulkan SDK (LunarG; Khronos Group, n.d.)
- A C++ Compiler (Linux: GCC (GCC, 2026), Windows: MSVC (Microsoft, 2026) oder MinGW (mingw-w64, 2026))
- CMake (kitware, n.d.)

The last two are only needed for compilation. The GitHub repository (Goth, LifeLike-Engine, 2026) contains the rest of the code and can be downloaded either manually or via a terminal command. Unfortunately, the libraries in `third_party` usually need to be downloaded separately, since they are just links to the respective repositories. After that, the program must be compiled (see Chapter 3.8). Alternatively, pre-compiled versions of the program are available under "Releases" (Goth, LifeLike-Engine Releases, 2026) in the GitHub repository.

3.8 Compiling and Executing

Updated information and instructions on compiling and running the program can be found in `README.md`.

The program uses CMake to compile the code and make it executable, but this currently only works on Linux. To keep things organized, a new folder for the compilation should be created. Starting from the main folder (which contains CMakeLists.txt), it can be compiled using the following terminal commands:

```
1. mkdir build && cd build
2. cmake ..
3. cmake --build . --config Release
```

If no errors occur, the compiled program should be located in the build folder under the name LifeLike-Engine. Some compilers, such as MSVC, the default C++ compiler on Windows, may place the file in a subfolder.

3.9 Future

Since the program will likely receive a few updates after its release, some elements may still change. Instructions for installation and use, as well as other important information, can be found in `README.md` and will be updated in a timely manner.

4 Capabilities of the Program

The simulation can model various cellular components and chemical compounds in terms of their shape and approximate behavior. In some cases, hydrophobic and hydrophilic interactions based on the Lennard-Jones potential are sufficient to give rise to more complex structures composed of multiple compounds. In addition, more complex interactions can be enabled by allowing any number of actions to occur when two compounds come into contact. Here are some examples that have been implemented and tested:

4.1 Phospholipid

The first and simplest examples of a compound are phospholipids, which, under the right conditions and after some time, can come together to form short sections of a double membrane.

Typ 1

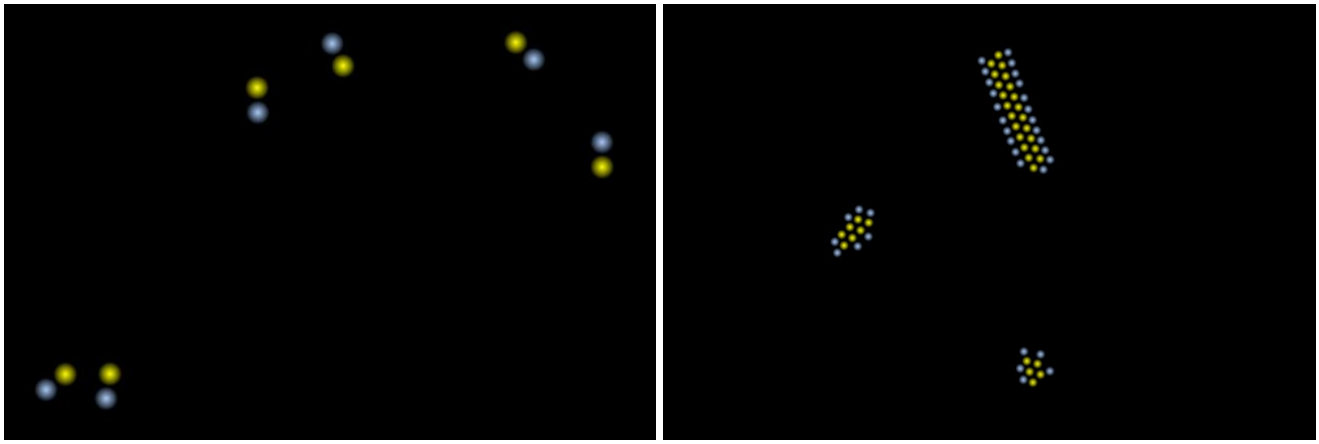


Figure 6: (Left) Individual type 1 phospholipids. (Right) A chain of phospholipids

Each phospholipid consists of a hydrophilic (blue) and a hydrophobic (yellow) particle. Strong hydrophobic interactions cause groups of phospholipids to form, with their hydrophobic ends facing inward and their hydrophilic ends facing outward. With the correct values for the forces between hydrophobic-hydrophobic, hydrophobic-hydrophilic, and hydrophilic-hydrophilic interactions, as well as for the distance at which these forces occur, longer segments consisting of two rows of phospholipids form. As in a typical cell membrane, the hydrophilic ends of both rows face outward. However, these segments usually remain short and cannot form a longer chain. The reason for this is likely the limited freedom of movement in two-dimensional space. As a result, the phospholipids at the ends of the chains are not oriented the same way as the rest of the chain; they can rotate relatively freely and are often aligned in such a way that they block

access to the hydrophobic inner part of the chain. Consequently, two “open” chains almost always repel each other, even if their ends touch. One possible solution would be to reduce the distance over which the attractive forces act; however, this also causes isolated phospholipids to stick to random points along the chain and destabilize its structure until it collapses and the phospholipids regroup into smaller clusters or large, disordered complexes. Strengthening the hydrophilic-hydrophilic attraction would partially align the lipids at the ends of the chains in the correct orientation; however, this also leads to the reformation of larger clumps, in which the structure of the chains is destroyed.

Typ 2

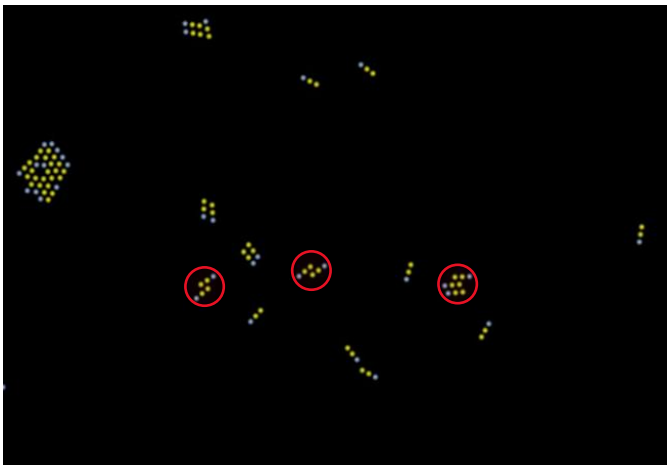


Figure 7: Type 2 phospholipids. Groups circled in red are no longer able to form a double membrane..

Another approach is to extend the phospholipids to prevent random or unintended orientation of individual lipids in a chain. For this reason, the program includes a second phospholipid that has two hydrophobic and one hydrophilic particle. The two hydrophobic groups also exert a stronger force than a single one would. However, tests have shown that the two-dimensional nature of the system causes some problems here. The biggest problem is the geometrically optimal arrangement of two of these phospholipids. This arrangement makes it less likely that a double membrane will form and also prevents the formation of a membrane with a smooth surface. The reason for this is, on the one hand, that once two lipids have attracted each other, they can practically no longer be separated due to the stronger attraction; and on the other hand, that due to their geometry, two lipids never position themselves perfectly next to each other but are always slightly offset. Therefore, lipids that lie next to each other in opposite orientations are practically unusable, and the lipids in chains are so strongly bound to one another that the chain can break rather than bend when subjected to external forces. For these reasons, Type 2 phospholipids seem to be unsuitable for a functioning cell membrane.

4.2 DNA/RNA

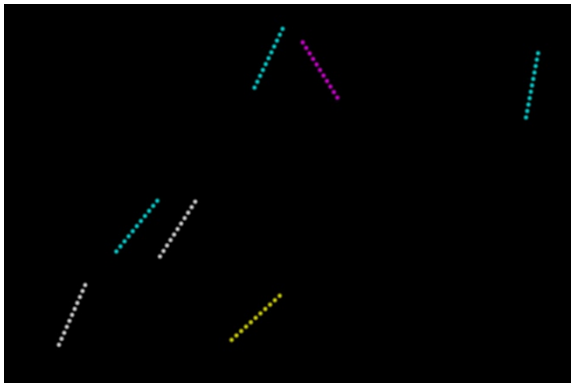


Figure 8: Examples of different DNA and RNA strands

Another simple compound consists of DNA and RNA strands, both of which can have exactly the same structure. By default, the simulation already includes a system that simulates interactions between DNA/RNA and proteins to enable transcription and translation. However, this system is only one of many possible approaches, and the following descriptions of transcription and translation refer only to this one method. The order of the particles in a DNA/RNA bond determines the direction of transcription and translation. Typically, the first particle is at one end of the strand and the last at the other, meaning that proteins move along it in this direction. Once the end is reached, a new bond is formed. Each DNA/RNA strand has a list of bonds that describes it. Upon completion of transcription/translation, a random element from this list is selected and synthesized. Thus, based on the composition of the list, a probability for a specific product can be determined, which allows, for example, the simulation of translation errors or the formation of defective proteins with a certain probability. By adjusting the spatial structure of the strand, additional interactions can be enabled or prevented. For example, a gap that is large enough would make it less likely for small proteins to reach the end of the strand, since they would be temporarily separated from it.

4.3 Protein

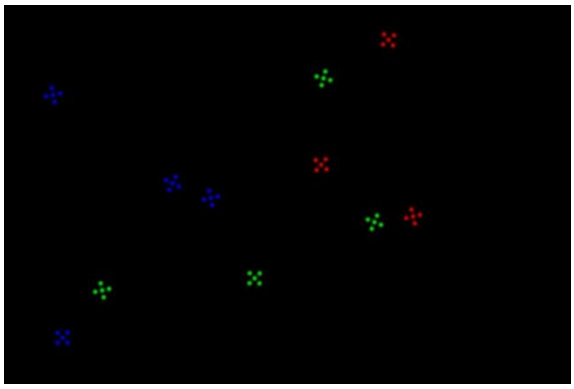


Figure 9: Examples of proteins with different functions

The last type of compound tested involves proteins, which enable the most complex interactions. By default, the simulation includes proteins that facilitate transcription and translation. Here, too, many different implementations are possible, and the following is just one of them: When a protein touches the first particle of a compound with which it interacts, it adjusts its speed to remain in contact with the compound. As long as this is the case, it moves slowly toward the next particle and continues on until the end of the compound. When it touches the last particle, this is considered successful transcription/translation, and a new compound is created.

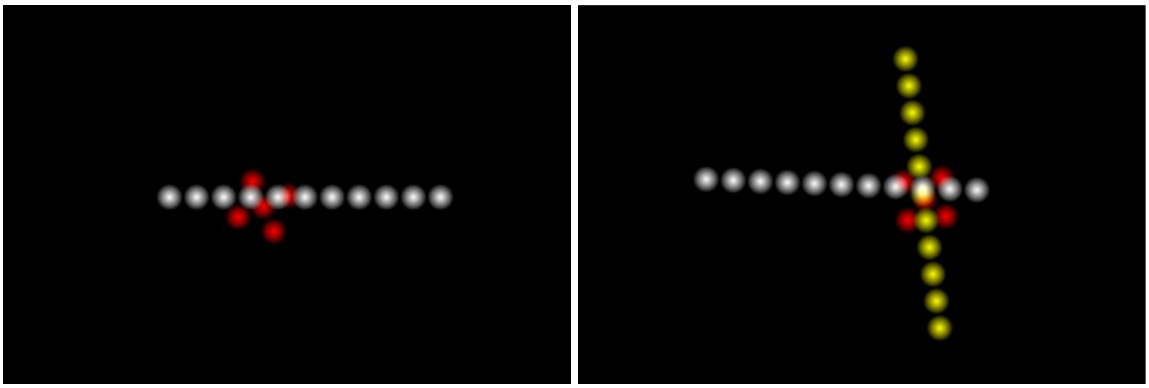


Figure 10: (Left) A protein (red) carrying out transcription on a strand of DNA. (Right) When transcription is complete, a strand of RNA (yellow) is produced.

4.4 Usage

Compounds can be modified or created in the `compounds.json` file in JSON format or in the `compounds_templates` list in `assets/compound_templates.cpp`. It is important that `compounds.json` is located in the folder where the program looks for it. By default, this is `templates`, which is located in the same folder as the program. Special interactions can be added to the second list further down in `assets/compound_templates.cpp`. By default, there are already examples for both, and at the end of each list there is a comment explaining the structure of an element to be added. To apply changes in `assets/compound_templates.cpp`, the program must be recompiled. However, as long as nothing else in the code has been changed, this shouldn't take long. Now a new or previously saved simulation can be started. If a saved simulation contains compounds, their interaction options must still be included in the list under the same name; otherwise, the simulation cannot be loaded. The list of compounds can be easily modified.

5 Limits/Problems

The simulation has many aspects that do not correspond to the real world.
As previously mentioned, it is severely limited by its two-dimensional nature, and most real-world processes do not really work in 2D. Therefore, they must be omitted or circumvented.
Another important problem is that the first law of thermodynamics is violated, since—even though the simulation represents a closed system—the total energy does not remain constant. This is primarily due to damping factors, which are intended to prevent excessively high velocities and rotations.
Another problem is that molecules lack elasticity and cannot easily change their structure. Thus, for example, the formation of chromosomes from a single strand of DNA is not possible.
A further limiting factor is the simulation time, which, as already mentioned, can vary. One possible solution would be to record the frames individually and combine them into a video based on the time elapsed in the simulation. If the simulated time per frame during recording is 0.001 seconds, this video would therefore need to have a frame rate of 1,000 frames per second.
Also, the computing power of the computer running the simulation is often a limiting factor. Depending on the computing power, the simulation slows down drastically as soon as several hundred particles are simulated.
Finally, even though the program can be used to a large extent without programming experience, it still takes some time to understand the various elements. Some of these, such as editing specific interactions, are also only possible with knowledge of C++.

6 Conclusion

The program generally meets the expectations I had at the start of the project. It can consistently and realistically simulate several thousand compounds and biophysical processes even on less powerful computers. During the approximately six-month development phase, I spent around 200 hours working on it according to my work journal, although this number could deviate by up to 25% from the actual time spent due to inaccuracies in my own estimates. This time has not only produced this program but also yielded a great deal of new knowledge and experience. However, there are also some goals that were not achieved. Most importantly, the program has never simulated a functioning cell. The largest structure tested so far was a closed cell membrane consisting of several hundred phospholipids. This is associated with an enormous, theoretically infinite number of possible compounds and interactions, of which only very few yield an interesting result. It takes a great deal of time and trial and error to stumble upon a functioning configuration. Compared to biology on Earth, several conclusions can be drawn from this. The most obvious is that the simulation is far too imprecise and simplistic to be compared to the real world. It is more interesting, however, to assume that at least some of the processes in the simulation correspond to those in reality. In this case, one could use it to explore the origins of biological life and, with enough patience, perhaps even create single-celled or multicellular life out of “nothing.” However, these are merely speculations, and I assume the program is far too slow to simulate such a process. Still, it would take only one successful attempt to create a cell to disprove this hypothesis. For me, the program is a good first step toward simulating microbiological systems. What has consistently amazed me while testing the program is that the Lennard-Jones potential alone is sufficient to simulate various intermolecular interactions. I plan to continue working on the program for some time after its release to complete its documentation, and I hope it serves as inspiration or a foundation for similar scientific projects.

7 Glossary

All explanations were generated by ChatGPT (OpenAI, n.d.).

- **API:** A defined interface through which different software components can communicate with one another.
- **Class**: A template that defines the properties and functions of a specific object type.
- **CMake:** A program for automating the creation and management of a software project's build process.
- **Compilation**: The process by which a compiler translates source code into executable or processable machine code.
- **Compiler:** A program that translates program code into a form that a computer can understand.
- **Debugging:** The process of finding and fixing errors in a program.
- **Declaration:** The statement that, for example, a variable, function, or class exists.
- **Definition:** The precise specification or implementation of what was previously declared.
- **Enum:** A data type that defines a fixed set of named values.
- **Frame:** A single image within a sequence of images that, for example, form an animation.
- **GitHub Repository:** A location on the GitHub platform for storing a project's files, source code, and version history.
- **GLFW:** A library for creating windows and processing input.
- **GLSL:** A programming language primarily used for programming shaders for graphics processing.
- **Header (C/C++)**: A file containing important declarations and information that can be used by other parts of the program.
- **Header-only library:** A library whose complete code is contained in header files and therefore does not require a separate implementation file.
- **Instance:** A specific object created based on a class.
- **Khronos Group**: An organization that develops open standards and interfaces for graphics, computing, and media, including, for example, OpenGL, Vulkan, and OpenCL.
- **Library**: A collection of pre-written program components that can be used in other programs.
- **Method**: A function that belongs to a class and describes the behavior of an object.
- **MSVC**: A development environment or compiler from Microsoft, primarily used to compile C and C++ programs into executable code.

- **Object-Oriented Programming (OOP)**: A programming concept in which data and associated functions are grouped into so-called objects.
- **SDL3**: A library for windows, input, audio, and graphics in programs.
- **SPIR-V**: A standardized intermediate format used, among other things, to represent and execute shader code on graphics cards.

8 Sources

8.1 Online Repository

There are two GitHub repositories related to this thesis. One contains the documentation and the work journal (Goth, Matura-Thesis, 2026), and the other contains the code (Goth, LifeLike-Engine, 2026). The latter is also linked in the former.

8.2 Websites

Bhargava, M., Gabai, D., Kochen, S., & Kohn, J. J. (n.d.). *John Horton Conway*. Retrieved October 08, 2026, from Princeton University: https://www.math.princeton.edu/people/john-horton-conway

DeepL. (n.d.). *German - English*. Retrieved October 10, 2026, from DeepL: https://www.deepl.com/en/translator

dujingning. (2026, May 01). *The INI header-only library for Modern C++ supports reading and writing, even writing comments. It is cross-platform and can be used on multiple operating systems.* Retrieved October 07, 2026, from inicpp: https://github.com/dujingning/inicpp

GCC. (2026, September 22). *GCC, the GNU Compiler Collection*. Retrieved Oktober 08, 2026, from https://gcc.gnu.org/

Geelnard, M., & Löwy, C. (n.d.). *GLFW is an Open Source, multi-platform library for OpenGL, OpenGL ES and Vulkan development on the desktop.* Retrieved March 08, 2026, from GLFW: https://www.glfw.org/

Goth, J. (2026, October 11). *LifeLike-Engine*. Retrieved October 11, 2026, from GitHub: https://github.com/j-goth/LifeLike-Engine

Goth, J. (2026, October 11). *LifeLike-Engine Releases*. Retrieved October 11, 2026, from GitHub: https://github.com/j-goth/LifeLike-Engine/releases

Goth, J. (2026, October 11). *Matura-Thesis*. Retrieved October 11, 2026, from GitHub: https://github.com/j-goth/Matura-Thesis

Khronos Group. (n.d.-a). *Khronos royalty-free open standards for 3D graphics, Virtual and Augmented Reality, Parallel Computing, Machine Learning, and Vision Processing*. Retrieved October 08, 2026, from Khronos Group: https://www.khronos.org/

Khronos Group. (n.d.-b). *Vulkan Home*. Retrieved October 08, 2026, from Vulkan: https://www.vulkan.org/

kitware. (n.d.). *CMake: A Powerful Software Build System*. Retrieved October 08, 2026, from CMake: https://cmake.org/

libsdl-org. (2026, October 11). *SDL*. Retrieved October 08, 2026, from https://github.com/libsdl-org/SDL
libsdl-org. (n.d.). *Simple DirectMedia Layer 3.0*. Retrieved June 26, 2026, from SDL Wiki: https://wiki.libsdl.org/SDL3/FrontPage
LifeWiki contributors. (2025, Dezember 15). *Conway's Game of Life*. Retrieved October 08, 2026, from LifeWiki: https://conwaylife.com/wiki/Conway's_Game_of_Life
Lohmann, N. (2026, October 11). *JSON for Modern C++*. Retrieved October 07, 2026, from json: https://github.com/nlohmann/json
LunarG; Khronos Group. (n.d.). *Welcome to the community for the Vulkan SDK. You can download the latest Vulkan SDK and get SDK questions answered at this site.* Retrieved October 06, 2026, from Vulkan: https://vulkan.lunarg.com/home/welcome
Microsoft. (2026, August 21). *Install the Microsoft C++ (MSVC) Build Tools*. Retrieved October 08, 2026, from Microsoft Ignite: https://learn.microsoft.com/en-us/cpp/overview/acquire-msvc?view=msvc-170
mingw-w64. (2026, October 03). *mingw-w64*. Retrieved October 08, 2026, from mingw-w64: https://www.mingw-w64.org/
MIT. (n.d.). *The MIT License (MIT)*. Retrieved October 08, 2026, from mit-license: https://mit-license.org/
Mohr, T. (n.d.). *About Me*. Retrieved August 30, 2026, from Particle Life Simulator: https://particle-life.com/about/tom-mohr.html
Mohr, T. (n.d.). *Particle Life Simulator*. Retrieved August 30, 2026, from Particle Life Simulator: https://particle-life.com/
OpenAI. (n.d.). *ChatGPT*. Retrieved October 11, 2026, from ChatGPT: https://chatgpt.com/
Overvoorde, A. (2016). *A tutorial that teaches you everything it takes to render 3D graphics with the Vulkan API. It covers everything from Windows/Linux setup to rendering and debugging.* Retrieved April 04, 2026, from Introduction - Vulkan Tutorial: https://vulkan-tutorial.com/
Ventrella, J. (2025, Februar). *An asymmetrical particle system*. Retrieved August 30, 2026, from Clusters: https://www.ventrella.com/Clusters/
Ventrella, J. (n.d.). *jeffrey ventrella*. Retrieved August 30, 2026, from jeffrey ventrella: http://www.jjventrella.com/
vit-vit. (2015, June 12). *Modern and efficient C++ Thread Pool Library*. Retrieved August 25, 2026, from CTPL: https://github.com/vit-vit/ctpl
Wikipedia contributors. (2026, September 14). *Lennard-Jones-Potential*. Retrieved August 31, 2026, from Wikipedia – Die freie Enzyklopädie: https://en.wikipedia.org/w/index.php?title=Lennard-Jones_potential

8.3 Image Sources

Fig. 1: https://media.geeksforgeeks.org/wp-content/uploads/20240318035545/game-660.png

Fig. 2: https://hunar4321.github.io/particle-life/images/big_pic.jpg

Fig. 3: Self-taken image

Fig. 4: Self-made formula

Fig. 5: https://de-academic.com/pictures/dewiki/49/12-6-Lennard-Jones-Potential.png

Fig. 6–12: Self-taken image

8.4 AI

AI Model	Usage	Location
DeepL (DeepL, n.d.)	Translate the original German documentation into English	This entire version of the documentation
ChatGPT (OpenAI, n.d.)	Explaining the Parameters of the Lennard-Jones Potential	Chapter 3.5, paragraph following Fig. 4
ChatGPT (OpenAI, n.d.)	Explaining the terms in the glossary	Chapter 7, Glossary

9 Acknowledgments

I would like to thank Dr. Andreas Leuthardt for supervising me during the project phase, thereby making this high school thesis possible, and for helping me improve both the program and the documentation.

I would also like to thank Mr. Adrian Kaiser for his help in revising my documentation and for the many helpful tips that improved my program.

Finally, I would like to express my sincere gratitude to my parents, Bettina and Markus Goth, for proofreading the documentation during the last week before submission and for their many suggestions for improvement.

10 Statement of Integrity

I, Jonathan Goth, 4A,

hereby confirm with my signature that the submitted paper was written independently and without unauthorized assistance from third parties. My exploration of the topic was based exclusively on my own work and research. No unauthorized aids were used. I confirm that I have fully listed all sources and informants used in the bibliography or in another section designated for this purpose. All quotations and paraphrases (indirect quotations) have been identified and cited. If I have used information from an AI system such as ChatGPT, I have correctly identified it as such in my Matura thesis in accordance with the guidelines in the Matura Thesis Handbook, including how and for which questions the AI was used. I confirm that the printed copy of the Matura thesis is identical to the digital version. I am aware that the entire thesis or parts thereof may be checked using appropriate software to detect plagiarism or AI-generated text.

Date: October 11, 2026, Signature: 